

Milestone 2 Deliverable

Part 1: Methodology – Technical details of models/tools applied.

Changes from Milestone 1

Our project investigates whether prompt-engineered LLMS can effectively classify financial news sentiment, and whether few-shot prompting improves performance compared to zero-shot prompting. In Milestone 1, we chose to use the Financial News Market Events Dataset from Kaggle to build a multi-dimensional risk tagger. During implementation, we discovered that the dataset contains only 50 unique headlines repeated approximately 57 times each with randomly assigned labels, making it unusable as ground truth. We switched to the Financial PhraseBank, a peer-reviewed benchmark with 4,840 unique financial sentences annotated by 16 finance professionals. We use the AllAgree subset (2,264 sentences) and sampled 200 for evaluation. Our research question, does few-shot prompting improve over zero-shot, remains the same, but the scope narrowed from multi-dimensional tagging to sentiment classification.

Models and Tools Applied

We applied three models to compare performance across different approaches:

- Sklearn Baseline (TF-IDF + Logistic Regression): We used scikit-learn's TfidfVectorizer with 5,000 max features and (1,2) ngram range to convert sentences into numerical vectors. Then trained a Logistic Regression classifier with balanced class weights to handle the class imbalance (61% neutral, 25% positive, 13% negative). This serves as a traditional ML baseline to contextualize the LLM results. We also computed a majority-class baseline (always predicting "neutral") as a lower bound.
- LLM Zero-Shot (GPT-40-mini): We used OpenAI's GPT-40-mini with temperature=0 to classify each sentence with no labeled examples. The prompt instructs the model to classify sentiment from an investor's perspective and return a JSON response with the predicted sentiment.
- LLM Few-Shot (GPT-40-mini): Same model and instructions, but the prompt includes 6 labeled examples (2 positive, 2 negative, 2 neutral) drawn from the dataset but excluded from the test set.

Pipelines

- Preprocessing: We loaded the Financial PhraseBank AllAgree subset (2,264 sentences) and parsed the @-separated format. No additional text cleaning was needed since the dataset is already well-formatted.
- Train/Test Split: For the Sklearn baseline, we use the 80/20 stratified split (1,811 train/453 test). For the LLM evaluation, we sampled 200 sentences with stratified sampling to preserve class proportions, since each sentence requires an API call.
- Tokenization: TfidfVectorizer handles tokenization internally for the Sklearn baseline using unigrams and bigrams. For the LLM pipeline, sentences are passed directly as text in the prompt since GPT-40-mini handles tokenization internally.

- **Model Training:** Logistic Regression was trained on TF-IDF vectors from the training set. The LLM requires no training as it uses in-context learning through prompt engineering, which is a key advantage of this approach.
- **Evaluation:** All model predictions were compared against human annotated ground truth labeled using accuracy, precision, recall, F1-score, and confusion matrices.

Evaluation Metrics

We used the following metrics computed via scikit-learn:

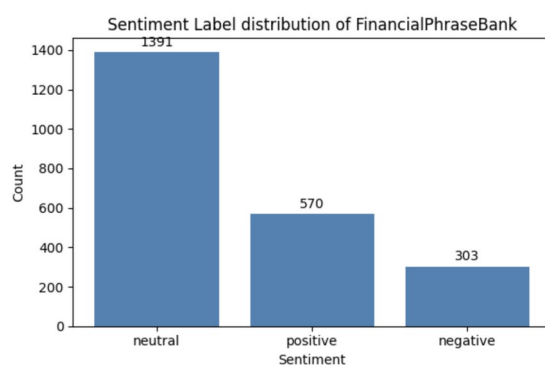
- **Accuracy:** Overall percentage of correct predictions across all classes.
- **Precision:** For each class, the fraction of predicted labels that were correct. Important because a model that overpredicts “neutral” would have low precision on that class.
- **Recall:** For each class, the fraction of actual labels that were correctly predicted. Important for detecting whether the model misses negative or positive sentences.
- **F1-score:** Harmonic mean of precision and recall. We reported both per class F1 and weighted average F1 to account for class imbalance.
- **Confusion matrix:** Visualized as heatmaps to show exactly which classes are being confused with each other.
- **Perplexity:** We did not use perplexity because it measures how well a language model predicts text, which applies to generative tasks. Our task is classification, so the ones above are the appropriate metrics.

Part 2: Results – Clear, concise presentation

EDA Visualization

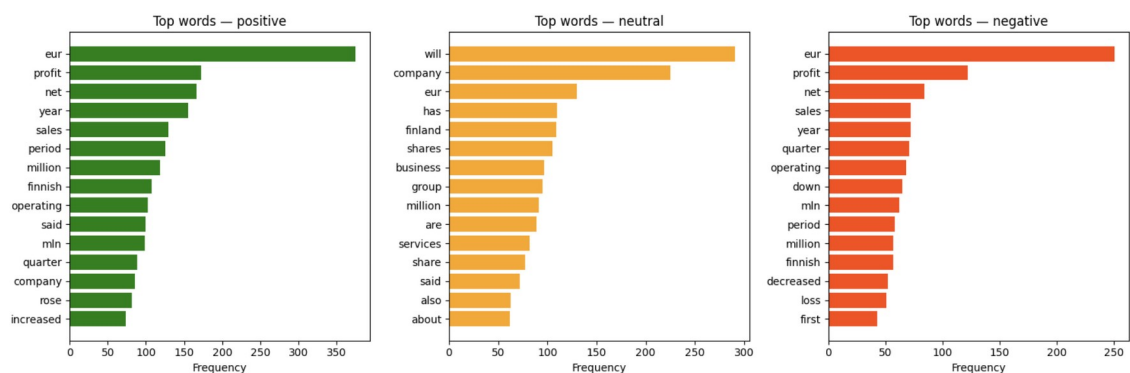
For the EDA part, we had three visualizations - sentiment label distributions, top key words per class and heatmap of confusion matrix.

- **Sentiment label distribution**



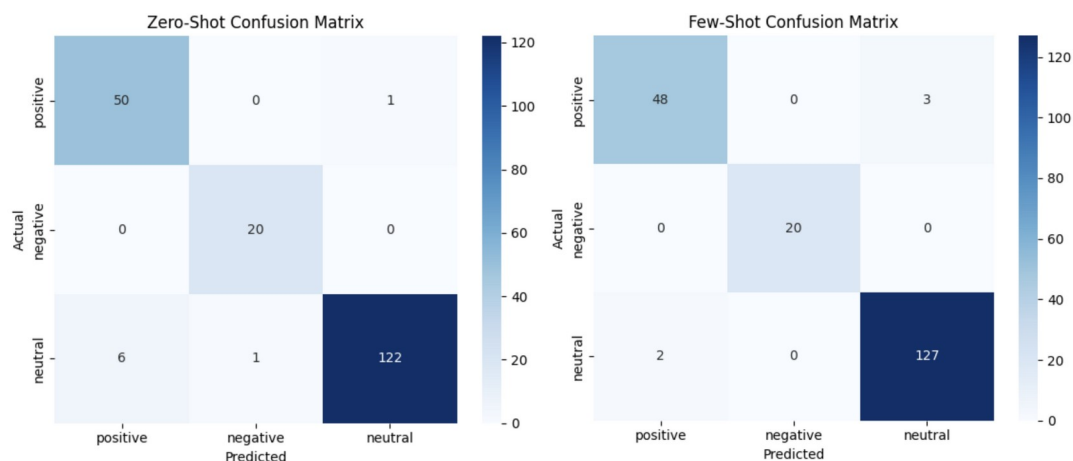
The dataset shows a class imbalance, neutral label domains about 61.5% of the full dataset, with positive labels at ~25% and negative labels at ~13%. So, we used `class_weight='balanced'` in the logistic regression baseline and macro f1 as the primary metric.

- **Top key words per class**



We evaluated the top words appearing in the headline for each class to see if the headline reveals any lexical pattern. By eliminating the filling words, the result shows that ‘increased’, ‘period’, ‘rose’ appears more frequently in positive headlines and ‘quarter’, ‘down’, ‘decreased’, ‘loss’ are dominative in negative headlines. Top key words in neutral headlines (‘net’, ‘sales’, ‘company’, ‘reported’) are more neutral and general, without directional keywords. These patterns exhibit explicit sentiment signals in headlines; this can explain the high accuracy of TF-IDF.

- Heatmap of confusion matrix(zero-shot and few-shot)



Two heatmaps reveal a directional error pattern: the dominant failure mode in zero-shot is neutral sentences predicted as positive. Few-shot prompting substantially reduces this error class, at the cost of introducing a small number of new false-neutral predictions for genuinely positive sentences.

Quantitative results

- Sklearn baseline: Always predicts “neutral” and gets ~64.5% accuracy. This is because about 61.5% of the dataset is neutral. It has poor macro f1 accuracy(~0.34), which means it fails on positive/negative classes.
- TF-IDF+Logistic Regression: reaches accuracy with ~90% and macro F1 with ~0.87. The macro F1 improves because sentiment is often related to keywords, but the transitional model struggles with context and nuanced meaning

- Zero-shot LLM: For zero-shot LLM, no training and examples are provided. It has 96% accuracy and 0.961 macro F1. The model captures context, meaning, and financial reasoning and has better performance than TF-IDF.
- Few-shot LLM: The model is given 6 labeled examples (2 per class) before asking for the prediction. Few-shot accuracy is 97.5% and macro F1 is 0.97, which is slightly improved and reaches the best performance over all models.

Part 3: Analysis – Interpret results; negative results are acceptable with justification.

- The macro F1 gap between TF-IDF+LogReg (0.868) and zero-shot LLM (0.961) is 9.3 points, and few-shot extends this to ~10.2 points. These gaps are relative to the task difficulty and hold despite the TF-IDF model benefiting from `class_weight = 'balanced'` and a larger evaluation set ($n=453$ vs. $n=200$)
- TF-IDF has 0.83 F1 scores on both negative and positive classes, compared to the LLM's 0.975 and 0.935 respectively. One of the reasons could be sentences like 'growth margin slowed down due to the financial crisis' contain the positive term 'growth' and the negative term 'slowed down'. TF-IDF partially captures this, but GPT-4o-mini correctly parses the qualifier as modifying growth negatively. This explains the TF-IDF model having a lower recall value on positive (0.82) compared with 0.98 for LLM. The few-shot confusion matrix confirms that TF-IDF's residual errors follow a clear lexical pattern rather than random noise. The other reason could be that TF-IDF treats each vocabulary frequency in the headline independently, it cannot resolve financial implications in the headline without contextual reasoning.
- The 6 labeled examples calibrate the model to the Financial PhraseBank annotation convention: sentences that describe financial magnitude, corporate process, or factual reporting without a directional outcome are labeled neutral from an investor perspective, even when they contain words like 'sales' or 'costs' that carry implicit valence in isolation. This calibration effect is more important than the examples' surface content. The few-shot improvement comes with a trade-off: 2 new errors were introduced (rows 1 and 38) where positive sentences like 'Re-use back into PET bottles has steadily increased...', 'Net sales will, however, increase from 2005' were relabeled neutral. This suggests the examples may have overcorrected the model toward neutral conservatism, causing it to discount weak positive signals.
- Across both models, the single most common error type is neutral sentences predicted as positive. Out of 8 zero-shot errors, 6 (75%) follow this pattern. Of the 5 remaining few-shot errors, 3 are still neutral→positive, 1 is positive→neutral (overcorrection), and 1 is neutral→negative. This asymmetry reflects a systematic model tendency to interpret any mention of revenue, sales, or operational activity as positive rather than informational.

- Class imbalance: The majority-class baseline achieves 64.5% accuracy, which is higher than it appears reasonable for a ‘random’ model. This is because 61.5% of sentences are neutral. This baseline's macro F1 is approximately 0.34, exposing the deception. For all model comparisons, macro F1 is therefore the primary metric. The TF-IDF model's jump from 0.90 accuracy to 0.868 macro F1score and the majority baseline's drop from 0.645 accuracy to 0.34 macro F1 illustrate this gap concretely.
- It can be observed that few-shot model achieves strong performance, with accuracy, precision, recall, and F1-score all around 0.98. This indicates that the model is highly effective at classifying financial sentiment when provided with examples. This represents a clear improvement over the zero-shot results of 0.96, indicating that the inclusion of labeled examples enhances the models ability to classify financial sentiment. The improvement suggests that few-shot prompting helps the model better capture subtle distinctions, particularly in cases where sentiment is implicit or context dependent. By providing concrete examples, the model is able to align its predictions more closely with domain specific interpretations of sentiment.
- After comparing misclassified examples between zero-shot and few-shot prompting, we observe that the few-shot model tends to classify more headlines as neutral, even in cases where the zero-shot model predicts positive or negative sentiment. This suggests that the few-shot examples encourage a more conservative interpretation of sentiment, treating factual financial statements as neutral unless clear positive or negative indicators are present. As a result, the few-shot model may reduce over-classification of sentiment but can also miss weaker signals of positivity or negativity.

Part 4: Plan for Additional Analysis – New methods, datasets, or simulations, with justification.

Our current results demonstrate strong performance on Financial PhraseBank, but this dataset consists of formally written, carefully annotated sentences from financial reports. Real financial news is messier and more informal, which raises the question of whether our pipeline generalizes beyond a clean benchmark. The broader goal of a project like this is to support a real-world application, such as a live sentiment dashboard where a user queries a company name, the app retrieves recent headlines from a news API, and the pipeline classifies each one in real time. Achieving this requires a pipeline that can handle noisy, real-world text, not just well-formatted annotated sentences. Our Milestone 2 results validate the approach on a controlled benchmark, but further analysis is needed before we can confidently extend this to a live setting.

- Chain-of-Thought Prompting
 - Looking at our error analysis, the most common mistake both models made was labeling neutral sentences as positive. This happened in 6 out of 8 zero-shot errors. Our guess is that the model is picking up on words like "sales" or "revenue" and assuming they are positive without actually thinking through whether the sentence has a real directional signal for an investor. We want to

test chain-of-thought prompting, where we ask the model to explain its reasoning before giving a label. We think this could reduce those neutral→positive errors by forcing the model to slow down and evaluate the sentence more carefully. If it does not help, that is still useful to know because it would suggest those errors are just the ceiling for this task.

- Testing on FiQA
 - We also want to test our pipeline on FiQA, a dataset that includes financial sentiment from news articles and microblogs. It is more informal and noisy than FPB, which makes it much closer to the kind of text a real dashboard would be processing. If our accuracy drops a lot on FiQA, we know we need to refine our prompts before building anything real. If it holds up, we have much stronger evidence that the approach actually generalizes.

Chain-of-thought tells us if the model is actually reasoning correctly, and FiQA tells us if it works on realistic data. If both go well, we have a solid foundation to argue that this pipeline could realistically power a live financial news sentiment tool.

Part 5: Work Plan – Updated weekly plan.

In the coming weeks, we plan to complete two additional analyses to build toward a real-world application of our pipeline. In Week 1, we will set up the FiQA dataset, explore its structure, and sample a stratified evaluation subset. In Week 2, we will implement chain-of-thought prompting and run it on our existing Financial PhraseBank test set, comparing the errors directly against our zero-shot and few-shot results. In Week 3, we will run all three prompting strategies on FiQA to test how well our pipeline generalizes to noisier, more informal financial text. In Week 4, we will finalize the Milestone 3 report, clean up the GitHub repo with an informative README, and prepare the presentation.